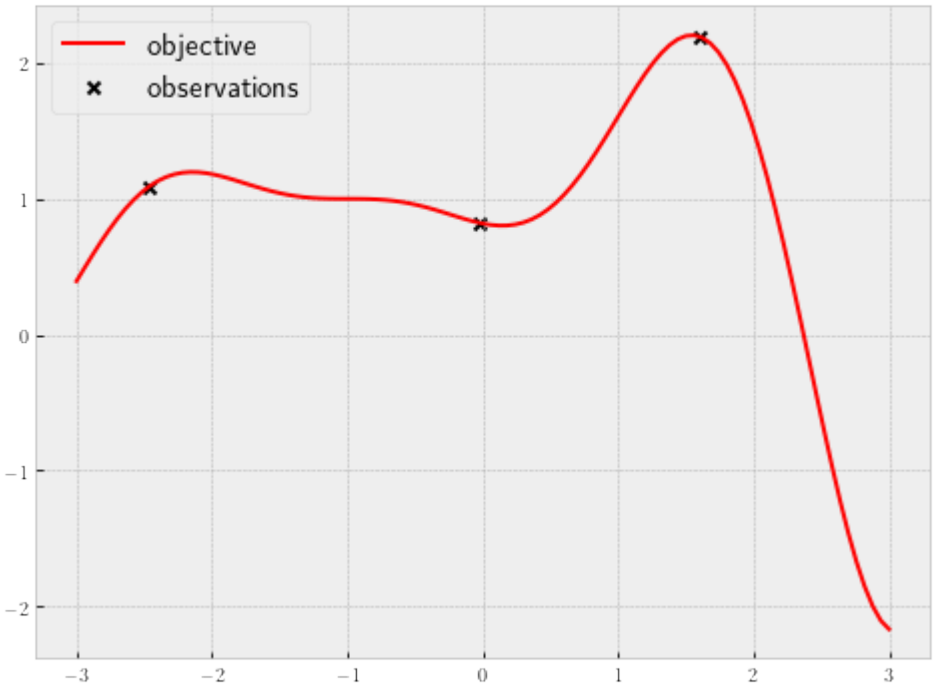


Figure 2.15 The objective function that is used in the current example, as shown by the solid line. The markers indicate points in the training data set.

The three markers that we see are points that we randomly select to include in our training data set. The locations of these training data points are stored in `train_x`, and their labels (thes value of the Forrester function at these locations) are stored in `train_y`. This sets up our regression task: implementing and training a GP on these three data points and visualize its predictions on the range between -3 and 3. Here, we have also created `xs`, which is a dense grid over this range.

2.4.2 Implementing a Gaussian process class

In this subsection, we will learn how to implement a GP model in Python. We will be using the GPyTorch library, the state-of-the-art tool when it comes to modern GP modeling.

> **IMPORTANT** **The design philosophy of GPyTorch is to follow the deep learning library PyTorch and to have all of its model classes extend a base model class. If you are familiar with implementing neural networks in PyTorch, you might know that this base is `torch.nn.Module`. With GPyTorch, we typically extend the `gpytorch.models.ExactGP` class.**

To implement our model class, we write the following code:

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>